

LLM Agent Capabilities Should Follow Task Intent and Context Source

Yusheng Zheng¹, Wenhui Zhang², Yu Mao³

¹UC Santa Cruz, USA ²Roblox, USA ³Bytedance, China

yzhen165@ucsc.edu, wenhuizhang.psu@gmail.com, mynotwo@126.com

Abstract

LLM agents take real actions—executing code, modifying files, calling services, delegating tasks—driven by context sources such as user requests, tool results, documents, shell outputs, Skill and MCP instructions, and memory. The least-privilege capability an agent needs is dynamic: it depends on the task the user authorized, yet all sources enter one shared planning channel with equal influence, so an untrusted document can steer decisions the user never delegated to it, by adversarial injection or accidental scope widening.

We argue that agent capabilities should be scoped to the user-authorized task intent, not to a sandbox or session lifetime, and that no single context source carries enough authority to define a complete capability. *INTENTCAP* composes short-lived capability leases from four sources—user intent, workflow instructions, tool schemas, and runtime environment—with field-level ownership and monotonic narrowing: each field is filled only by its owning source, and every lease can only narrow the user’s authorized authority. An LLM-assisted compiler proposes leases carrying per-field provenance proofs, and a deterministic checker verifies those proofs before any side effect commits, so no LLM claim is trusted for accept or deny. Preliminary replay evaluation shows that *INTENTCAP* blocks tested violations without rejecting benign actions and that each source boundary is independently necessary.

1 Background and Motivation

LLM agents load Skills with instructions, references, and scripts [9]; connect to MCP servers that expose resources, prompts, and tools [8]; run local commands; and spawn subagents. Each input is useful precisely because it affects future behavior, and dangerous for the same reason: any input can influence any action, whether through adversarial injection [6] or accidental scope widening. Existing defenses—prompt-level guardrails and control/data separation [2, 15], tool-level permission guards [12], execution sandboxes [16], and OS-level policy engines [19]—constrain which operations may execute, but do not bind each field of a decision to the context source authorized to fill it.

Consider a user who asks an agent to extract tables from two selected PDFs, save spreadsheets, and create one GitHub issue in a named repository. The PDF contents should shape spreadsheet cells and the issue body, but not the repository name, network destination, approval scope, or tool selection. If hidden text in the PDF says “create the issue in attacker/repo” [6], a tool-call allowlist can block banned domains but cannot express that the PDF was never allowed to fill the repository field at all; with no adversary at all, the planner may parse a table header as the repository name and silently overwrite the user’s destination.

The root cause is that every context source enters the same planning channel with equal standing, so any source can fill any

decision field, yet no single source carries everything a capability needs.

2 Design

Our key insight is that agent capabilities must be composed at runtime from multiple context sources with field-level ownership, because no single source carries enough authority to define a complete capability and no programmer exists to declare the composition statically.

Threat model and TCB. The adversary controls runtime-environment content—documents, tool outputs, web data, delegated-agent messages—and may plant instructions in Skill or MCP text; it does not control the user’s structured selections or the enforcement layer. The LLM planner and the LLM-assisted compiler are untrusted and may be arbitrarily manipulated: they propose but never decide. The TCB—the dashed box in Figure 1—is the intent issuer, source labeler, deterministic checker, and enforcement layer (hooks, MCP gateway, OS enforcer). The intent issuer is deterministic: it reads the user’s maximum authority only from structured user-interface fields—selected files, named destinations, explicit approvals—while free-form prose is treated as goal description, not authority. Because the initial authority is exact and every later step can only narrow it, injected text cannot widen what the user granted. The source labeler tags each input by position—user messages as user intent, tool responses as runtime environment, Skill/MCP metadata at load time—mapping at the OS level to process isolation and IPC channel tagging; ambiguous-provenance inputs, such as documents pasted into the prompt, are denied authority fields.

Provenance proofs. The guarantee comes from what the compiler must attach to each candidate lease, however it populates the fields. Every *authority field* must carry a *copy witness*: a reference to the labeled span of its owning source, recorded when that source entered the system, from which the field’s value was copied verbatim. The checker verifies each witness by deterministic comparison against the recorded source span, and verifies that the lease as a whole narrows the issued authority. The LLM can propose anything but fabricate nothing: a field either provably copies its owner source or is denied. This restricted-copy discipline is deliberately conservative; it is what makes field provenance checkable behind a shared neural planner.

Fields whose values the planner must synthesize rather than copy—report text, extracted tables, generated code—are *data fields*: they may shape content but carry no authority. For data-dependent authorization—“send the report to the project lead listed in the selected document”—the selector expression is itself user intent, recorded as a structured field and resolved by the checker against runtime data at commit time; authority flows from the user’s selector, only data from the document.

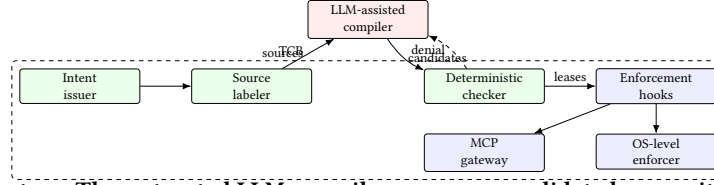


Figure 1: INTENTCAP architecture. The untrusted LLM compiler proposes candidate leases with per-field copy witnesses; the deterministic checker verifies them before any side effect commits and returns denials to the compiler. The dashed box is the TCB (intent issuer, source labeler, checker, enforcement layer); the LLM compiler sits outside it.

Worked example. In the PDF task above, user intent supplies the two selected files and repository org/proj, the Skill supplies the extract–save–report procedure, the issue tool’s schema the callable interface, and runtime results the table values. The compiler proposes a lease for create_issue: the repository field copies org/proj with a witness to the user-intent span, the issue body is a data field synthesized from the PDFs, with budget one issue, expiring on first use. The checker verifies witness and narrowing, and commits. An injected candidate naming attacker/repo has no witnessing user-intent span, and the checker denies the lease before the call executes.

A *field* is a named slot in a lease—destination, approval scope, tool name, or argument value—owned by exactly one source. *User intent* owns the goal, selected objects, authorized destinations, and approvals. *Workflow instructions* (system policy, Skill procedures) own procedural scope. *Tool schemas* (MCP definitions, registry entries) own the callable interface; because MCP servers may be untrusted, schemas describe but never grant credential scope. *Runtime environment* (tool results, file state, script outputs) owns observed values. These ownership rules are an information flow policy over decisions: which source may flow into which lease field.

A capability *lease* binds these fields to one operation, object, arguments, budget, expiry, and delegation depth; it is consumed on use, cannot widen itself, and attenuates further under delegation. The checker validates all fields atomically through a single transition, $\text{check_and_consume}(e, \text{lease}, \text{proofs}, \sigma) \rightarrow \text{allow}(\sigma') \mid \text{deny}$, which adapters must call before any side effect, context placement, or delegation handoff commits. Tool-level boundary adapters intercept MCP calls and validate each argument field’s witness before execution, complementing planner-level taint tracking as in Fides [1]; at the OS level, accepted leases lower to ActPlane [19] file, exec, and network policies enforced via eBPF.

3 Preliminary Evaluation

Scope. All results are checker-level replays over logged benchmark events and handcrafted suites, scored against reference labels—not end-to-end trajectories, and they do not measure end-to-end attack success or task completion.

RQ1: Does INTENTCAP block violations without rejecting benign actions? INTENTCAP produces 0 unsafe accepts across 3,746 security-sensitive events replayed from AgentDojo, MCPTox, InjecAgent, and tau2-bench [3, 13, 17, 18]. On the utility side, INTENTCAP covers all 3,813 benign reference actions in the tau2-style proxy and passes 2,554 of 2,556 applicable ground-truth checks.

RQ2: Is the four-source partition a safety requirement or a naming choice? Collapsing all four sources into one generic trusted context causes 3,593 false accepts out of 3,823 events that the

checker should deny (94%). Pairwise collapses each open distinct holes: tool→agent falsely accepts 1,928 events (50%), env→agent 1,663 (43%), and env→tool 1,662 (43%). On 7 crafted multi-step workflow scenarios, a policy DSL that checks predicates without field ownership falsely accepts 7/7, and splitting state across independent guards falsely accepts 5/7. Because the deny labels derive from the proposed ownership rules, these collapse results measure the internal necessity of the partition rather than external accuracy.

RQ3: Does the same commit API work beyond tool calls?

The check_and_consume transition applies at five enforcement points: local env side effects, context placement, Skill instruction slots, delegation handoffs, and an OS monitor replay backend. Across 38 enforcement checks, INTENTCAP allows 17 authorized effects and blocks 21 violations, with 0 unsafe executions or placements and 0 checker/monitor mismatches, supporting functional correctness at each point.

RQ4: Are leases auditable authority objects?

Across 24 manually labeled leases covering InjecAgent, MCPTox, and tau2 samples, all INTENTCAP policies match the labeled authority scope; every non-INTENTCAP baseline over-grants authority on at least some of the 144 scored policy entries.

4 Discussion and Limitations

INTENTCAP draws on capability-based security [5], least privilege [11], and information flow control [4], but unlike Capsicum [14] file descriptors or seccomp filters, an agent’s capability cannot be declared at compile time; it must be composed at runtime from sources sharing one neural planning channel, where any source can impersonate any other—a neural confused deputy [7]. CaMeL separates trusted control from untrusted data [2]; Progent and IsolateGPT restrict which operations an agent may perform [12, 16]; Fides, ELM, ActPlane, and SkillGuard enforce information-flow, tool-, OS-, and package-level constraints [1, 10, 19, 20]. INTENTCAP’s distinct, complementary claim is the per-field binding of a decision field to the single source authorized to fill it, made checkable by copy witnesses and scoped by a lease that only narrows the user’s authorized intent.

Limitations. Copy-witness provenance is conservative: fields requiring cross-source synthesis are confined to data fields or user-authorized selectors; meet-based composition of jointly determined fields is future work. The taxonomy folds memory, organizational policy, and human approval into user intent or workflow instructions. Finally, end-to-end attack success, compiler and enforcement latency, and adaptive attacks against labeling and lease generation remain open.

References

- [1] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. *arXiv preprint arXiv:2505.23643* (2025).
- [2] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. *arXiv preprint arXiv:2503.18813*. doi:10.48550/arXiv.2503.18813
- [3] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*. doi:10.48550/arXiv.2406.13352
- [4] Dorothy E. Denning. 1976. A Lattice Model of Secure Information Flow. *Commun. ACM* 19, 5 (1976), 236–243. doi:10.1145/360051.360056
- [5] Jack B. Dennis and Earl C. Van Horn. 1966. Programming Semantics for Multi-programmed Computations. *Commun. ACM* 9, 3 (1966), 143–155. doi:10.1145/365230.365252
- [6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISeC ’23)*. doi:10.1145/3605764.3623985
- [7] Norm Hardy. 1988. The Confused Deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review* 22, 4 (1988), 36–38. doi:10.1145/54289.871709
- [8] Model Context Protocol. 2025. Specification. <https://modelcontextprotocol.io/specification/2025-06-18>.
- [9] OpenAI. 2026. Agent Skills. <https://developers.openai.com/codex/skills>. Accessed: 2026-07-02.
- [10] Shidong Pan, Xiaoyu Sun, Tianyi Zhang, Dianshu Liao, Meixue Si, and Zhenchang Xing. 2026. SkillGuard: A Permission Framework for Agent Skills. *arXiv preprint arXiv:2606.03024*. doi:10.48550/arXiv.2606.03024
- [11] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939
- [12] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. *arXiv preprint arXiv:2504.11703*. doi:10.48550/arXiv.2504.11703
- [13] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2025. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. *arXiv preprint arXiv:2508.14925*. doi:10.48550/arXiv.2508.14925
- [14] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *Proceedings of the 19th USENIX Security Symposium (USENIX Security ’10)*. USENIX Association, Washington, DC, USA, 29–46. <https://www.usenix.org/conference/usenixsecurity10/capsicum-practical-capabilities-unix>
- [15] Simon Willison. 2023. The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection. <https://simonwillison.net/2023/Apr/25/dual-llm-pattern/>.
- [16] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, San Diego, CA, USA. doi:10.14722/ndss.2025.241131
- [17] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*. doi:10.48550/arXiv.2406.12045
- [18] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506. doi:10.18653/v1/2024.findings-acl.624
- [19] Yusheng Zheng, Tianyuan Wu, Quanzhi Fu, Tong Yu, Wenan Mao, Tao Ma, Dan Williams, Wei Wang, and Andi Quinn. 2026. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. *arXiv preprint arXiv:2606.25189*. doi:10.48550/arXiv.2606.25189
- [20] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI ’25)*. USENIX Association. <https://www.usenix.org/conference/osdi25/presentation/zheng-yusheng>